RESIBO, PLEASE

Public City Gallery API

Implementation & Submission Evidence

SUBMISSION REQUIREMENTS

1. Updated project source code.

2. Screenshots showing: API request implementation; retrieved data displayed in the application; JSON response sample.

3. Brief documentation including: API name; endpoint(s); request method(s); features added through API integration.

EXPECTED OUTPUT DELIVERED

- Publish → Retrieve → Browse → View using Supabase REST.
- Explicit HTTP GET, HTTP POST, jsonDecode, and jsonEncode.
- Sanitized public payload, RLS SQL, safe states, Retry, and offline cache.
- One PDF containing requirements, documentation, screenshots, code, and verification.

Prepared August 18, 2026

Fictional civic simulation — no real election guidance

Brief API Documentation

School API integration requirement

API Name & Purpose

Supabase REST API (PostgREST). It stores and retrieves sanitized fictional city snapshots for the Resibo, Please Visit Cities gallery.

Endpoints & Request Methods

- GET {SUPABASE_URL}/rest/v1/city_snapshots — retrieves up to 50 published cities ordered newest first.
- POST {SUPABASE_URL}/rest/v1/city_snapshots — publishes one completed sanitized city and requests the inserted row.

Data Format

JSON with city name, term number, starting pressure, top issue, selected candidate and party, final condition, outcome summary, major issues, major events, created date, and server ID.

Features Added

- Term Report publish confirmation with Publishing, Published, and Failed/Retry states.
- Visit Cities loading, successful, empty, no-internet, invalid-response, and server-failure states.
- Newest-first city cards, public detail page, Retry, and last-successful-response cache.
- Debug-only JSON response logging; no raw exceptions shown to players.

Security Boundary

The app accepts only a normal publishable/anon key through --dart-define. It never includes a service-role key, database password, entire save, world seed, hidden candidate stats, investigation history, or personal information.

Screenshot 1 — API Request Implementation

Actual source: GET, POST, JSON encoding and decoding

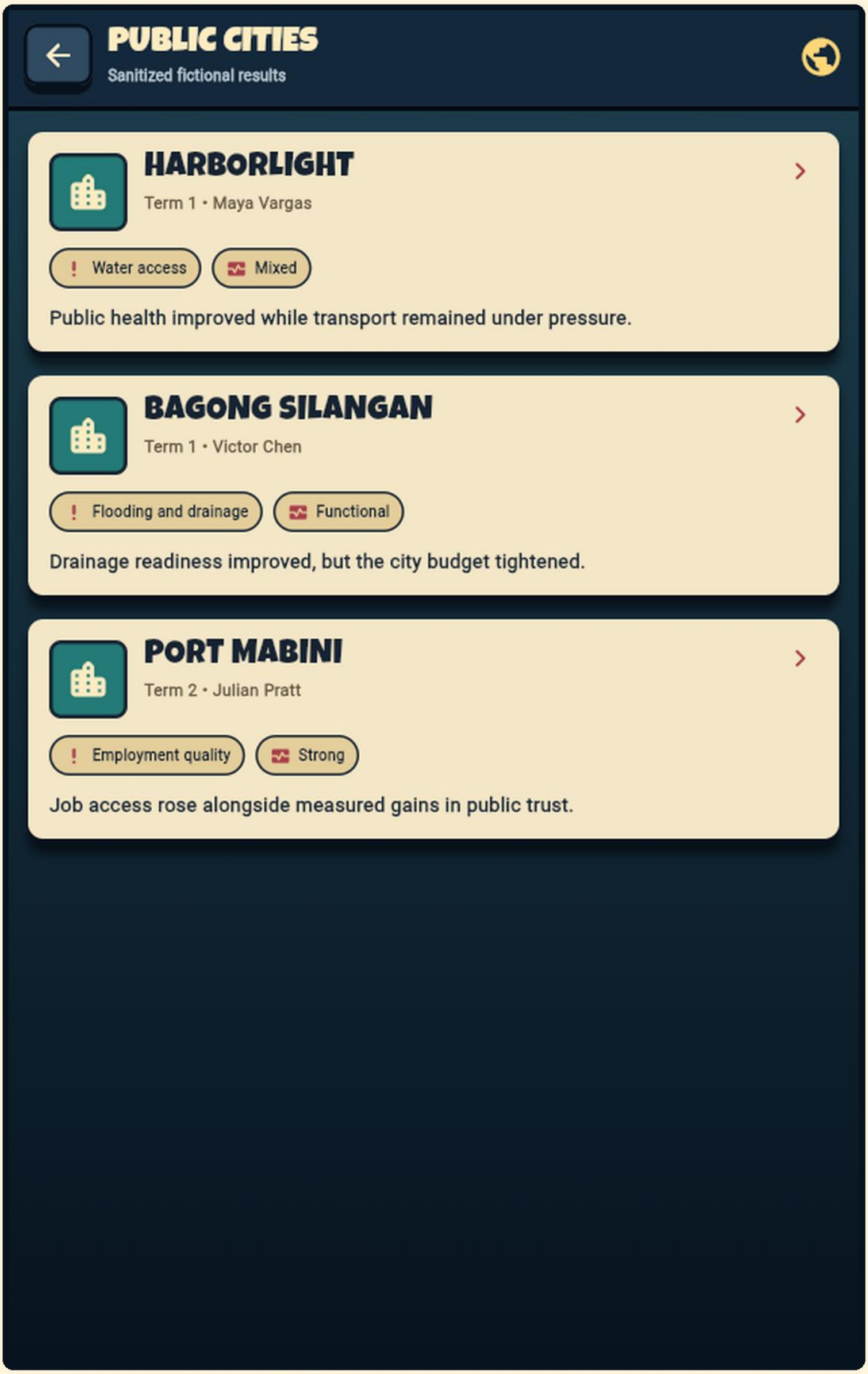
API REQUEST IMPLEMENTATION

lib/data/gallery/city_gallery_api_service.dart

```
76    Future<List<PublicCitySnapshot>> fetchSnapshots() async {
77      _ensureConfigured();
78      final uri = config.endpoint.replace(
79        queryParameters: const {
80          'select':
81              'id,city_name,term_number,starting_pressure,top_issue,'
82              'selected_candidate_name,candidate_party,final_city_condition,'
83              'outcome_summary,major_issues,major_events,created_at',
84          'order': 'created_at.desc',
85          'limit': '50',
86        },
87      );
88      try {
89        final response =
90            await (_client == null
91                    ? http.get(uri, headers: _headers) // HTTP GET
92                    : _client.get(uri, headers: _headers))
93                .timeout(const Duration(seconds: 12));
94        if (response.statusCode < 200 || response.statusCode >= 300) {
95          throw CityGalleryApiException(
96            CityGalleryFailure.server,
97            statusCode: response.statusCode,
98          );
99        }
100       final body = utf8.decode(response.bodyBytes);
101       _logJson('GET', body);
102       final decoded = _decodeJson(body); // JSON decoding
103       if (decoded is! List) {
104         throw const CityGalleryApiException(CityGalleryFailure.invalidResponse);
105       }
106       try {
107         return List.unmodifiable(
...
123
124   @override
125   Future<PublicCitySnapshot> publishSnapshot(
126     PublicCitySnapshot snapshot,
127   ) async {
128     _ensureConfigured();
129     try {
130       final headers = {..._headers, 'Prefer': 'return=representation'};
131       final body = jsonEncode(snapshot.toPublishJson()); // JSON encoding
132       final response =
133           await (_client == null
134                   ? http.post(
135                       config.endpoint,
136                       headers: headers,
137                       body: body,
138                     ) // HTTP POST
139                   : _client.post(config.endpoint, headers: headers, body: body))
140               .timeout(const Duration(seconds: 12));
141       if (response.statusCode < 200 || response.statusCode >= 300) {
142         throw CityGalleryApiException(
143           CityGalleryFailure.server,
144           statusCode: response.statusCode,
145         );
146       }
147       final responseBody = utf8.decode(response.bodyBytes);
148       _logJson('POST', responseBody);
149       final decoded = _decodeJson(responseBody);
150       if (decoded is! List || decoded.isEmpty) {
151         throw const CityGalleryApiException(CityGalleryFailure.invalidResponse);
152       }
153       try {
154         return PublicCitySnapshot.fromJson(_stringMap(decoded.first));
155       } on FormatException {
156         throw const CityGalleryApiException(CityGalleryFailure.invalidResponse);
157       }
158     } on CityGalleryApiException {
159       rethrow;
...
188   }
189
```

Screenshot 2 — Retrieved Data in Application

Compiled Flutter Web UI using the production HTTP service



Capture fixture note: no Supabase credentials are committed. For this proof image the same production service retrieved three rows from a local Supabase-compatible REST fixture.

Screenshot 3 — JSON Response Sample

Debug output from GET /rest/v1/city_snapshots

JSON RESPONSE SAMPLE

Debug output from local API capture fixture

```
[CityGalleryApi] GET JSON response:
[
  {
    "id": "8cbe205b-3304-4a6e-a28a-5e5ad8b27741",
    "city_name": "Harborlight",
    "term_number": 1,
    "starting_pressure": "strained",
    "top_issue": "Water access",
    "selected_candidate_name": "Maya Vargas",
    "candidate_party": "Bayhaven Civic Alliance",
    "final_city_condition": "mixed",
    "outcome_summary": "Public health improved while transport remained under pressure.",
    "major_issues": [
      "Water access",
      "Transport congestion",
      "Clinic capacity"
    ],
    "major_events": [
      "Mobile clinic response",
      "Water emergency"
    ],
    "created_at": "2026-08-18T09:30:00.000Z"
  },
  {
    "id": "04f20bb4-8bce-45ee-b74c-87ab51bc8205",
    "city_name": "Bagong Silangan",
    "term_number": 1,
    "starting_pressure": "crisis",
    "top_issue": "Flooding and drainage",
    "selected_candidate_name": "Victor Chen",
    "candidate_party": "Forward Bayhaven",
    "final_city_condition": "functional",
    "outcome_summary": "Drainage readiness improved, but the city budget tightened.",
    "major_issues": [
      "Flooding and drainage",
      "Public funds",
      "Employment"
    ],
    "major_events": [
      "Typhoon response",
      "Records review"
    ],
    "created_at": "2026-08-17T15:10:00.000Z"
  },
  {
    "id": "91543cf2-4753-4535-bb0d-ff83c287c4ce",
    "city_name": "Port Mabini",
    "term_number": 2,
    "starting_pressure": "stable",
    "top_issue": "Employment quality",
    "selected_candidate_name": "Julian Pratt",
    "candidate_party": "Workers and Enterprise Coalition",
    "final_city_condition": "strong",
    "outcome_summary": "Job access rose alongside measured gains in public trust.",
    "major_issues": [
      "Employment quality",
      "Cost of living"
    ],
    "major_events": [
      "Jobs compact",
      "Delivery milestone"
    ],
    "created_at": "2026-08-16T08:45:00.000Z"
  }
]
```

Relevant Source Code — Sanitized Snapshot

Dedicated model and completed-term builder

```
  Map<String, Object?> toPublishJson() => {
    'city_name': cityName,
    'term_number': termNumber,
    'starting_pressure': startingPressure,
    'top_issue': topIssue,
    'selected_candidate_name': selectedCandidateName,
    'candidate_party': candidateParty,
    'final_city_condition': finalCityCondition,
    'outcome_summary': outcomeSummary,
    'major_issues': majorIssues,
    'major_events': majorEvents,
    'created_at': createdAt.toUtc().toIso8601String(),
  };

  /// JSON used by the local public-gallery cache.
  Map<String, Object?> toJson() => {
    if (id != null) 'id': id,
    ...toPublishJson(),
...
PublicCitySnapshot buildPublicCitySnapshot({
  required City city,
  required CityRunConfiguration configuration,
  required TermResult result,
  required String? topIssueId,
  DateTime? createdAt,
}) {
  final rankedIssues = [...city.problems]
    ..sort((a, b) => b.issueWeight.compareTo(a.issueWeight));
  final topIssue = city.problems
      .where((problem) => problem.id == topIssueId)
      .map((problem) => problem.title)
      .firstOrNull;
  final events = <String>[];
  for (final phase in result.phases) {
    if (!events.contains(phase.title)) events.add(phase.title);
  }
  return PublicCitySnapshot(
    cityName: configuration.cityName.trim(),
    termNumber: city.term,
    startingPressure: configuration.startingPressure.name,
    topIssue: topIssue ?? 'Not recorded',
    selectedCandidateName: result.candidate.name,
    candidateParty: result.candidate.party,
    finalCityCondition: _conditionFromResult(result),
    outcomeSummary: result.summary,
    majorIssues: List.unmodifiable(
      rankedIssues.take(3).map((problem) => problem.title),
    ),
    majorEvents: List.unmodifiable(events.take(4)),
    createdAt: (createdAt ?? DateTime.now()).toUtc(),
  );
}

String _conditionFromResult(TermResult result) {
  var total = 0;
  for (final indicator in CityIndicator.values) {
    final raw = result.after.valueOf(indicator);
    total += indicator == CityIndicator.corruptionPressure ? 100 - raw : raw;
  }
  final average = total / CityIndicator.values.length;
  return switch (average) {
    >= 72 => 'strong',
    >= 60 => 'functional',
    >= 48 => 'mixed',
    >= 36 => 'strained',
    _ => 'critical',
  };
}
```

Privacy result: this builder has no field for seed, candidate capabilities, evidence views, private notes, or the save object, so POST cannot serialize them accidentally.

Relevant Source Code — Cache & UI States

Live retrieval, offline fallback, loading/error/empty/results

```
  Future<CityGalleryLoadResult> fetchSnapshots() async {
    try {
      final live = await remote.fetchSnapshots();
      _snapshots = _newestFirst(live);
      await cache.write(_snapshots);
      return CityGalleryLoadResult(snapshots: _snapshots, fromCache: false);
    } on CityGalleryApiException catch (error) {
      if (error.failure == CityGalleryFailure.invalidResponse ||
          error.failure == CityGalleryFailure.notConfigured) {
        rethrow;
      }
      final saved = _newestFirst(await cache.read());
      if (saved.isEmpty) rethrow;
      _snapshots = saved;
      return CityGalleryLoadResult(
        snapshots: saved,
        fromCache: true,
        liveFailure: error.failure,
      );
    }
  }

  Future<PublicCitySnapshot> publishSnapshot(
    PublicCitySnapshot snapshot,
```

```

  Widget _body(CityGalleryStrings strings) {
    if (_loading) {
      return Center(
        key: const Key('gallery_loading'),
        child: _StatePanel(
          icon: Icons.cloud_download_rounded,
          title: strings.loading,
          loading: true,
        ),
      );
    }
    if (_failure case final failure?) {
      return Center(
        key: const Key('gallery_error'),
        child: _StatePanel(
          icon: Icons.cloud_off_rounded,
          title: switch (failure) {
            CityGalleryFailure.notConfigured => strings.notConfigured,
            CityGalleryFailure.invalidResponse => strings.invalidResponse,
            _ => strings.unavailable,
          },
          actionLabel: strings.retry,
          onAction: _load,
        ),
      );
    }
    final result = _result!;
    if (result.snapshots.isEmpty) {
      return Center(
        key: const Key('gallery_empty'),
        child: _StatePanel(
          icon: Icons.location_city_rounded,
          title: strings.empty,
          body: strings.emptyBody,
          actionLabel: strings.retry,
          onAction: _load,
        ),
      );
    }
    return RefreshIndicator(
      color: ResiboColors.mutedRed,
      onRefresh: _load,
      child: ListView(
        key: const Key('gallery_results'),
```

Backend Setup & Verification

Table/RLS contract and completed checks

Supabase Table and RLS

```
create table if not exists public.city_snapshots (
  id uuid primary key default gen_random_uuid(),
  city_name text not null check (char_length(city_name) between 1 and 60),
  term_number integer not null check (term_number between 1 and 100),
  starting_pressure text not null check (
    starting_pressure in ('stable', 'strained', 'crisis')
  ),
  top_issue text not null check (char_length(top_issue) between 1 and 100),
  selected_candidate_name text not null check (
    char_length(selected_candidate_name) between 1 and 100
  ),
  candidate_party text not null check (
    char_length(candidate_party) between 1 and 120
  ),
  final_city_condition text not null check (
    final_city_condition in ('strong', 'functional', 'mixed', 'strained', 'critical')
  ),
  outcome_summary text not null check (
    char_length(outcome_summary) between 1 and 600
...

create index if not exists city_snapshots_created_at_idx
  on public.city_snapshots (created_at desc);

alter table public.city_snapshots enable row level security;

drop policy if exists "Public city snapshots are readable" on public.city_snapshots;
create policy "Public city snapshots are readable"
  on public.city_snapshots
  for select
  to anon, authenticated
  using (true);

drop policy if exists "Sanitized city snapshots are publishable" on public.city_snapshots;
create policy "Sanitized city snapshots are publishable"
  on public.city_snapshots
  for insert
  to anon, authenticated
  with check (
    char_length(city_name) between 1 and 60
    and term_number between 1 and 100
    and jsonb_array_length(major_issues) <= 8
    and jsonb_array_length(major_events) <= 8
  );

```

Configuration

Run with --dart-define=SUPABASE_URL=https://YOUR_PROJECT.supabase.co and --dart-define=SUPABASE_ANON_KEY=YOUR_PUBLISHABLE_OR_ANON_KEY. The SQL grants anonymous/authenticated SELECT and INSERT only; there is no public UPDATE or DELETE policy.

Verification Completed

- flutter analyze — no issues found
- flutter test — all 97 tests passed
- Focused API tests cover valid GET/POST, JSON parsing, invalid response, safe server failure, loading, empty, Retry, cache, detail navigation, sanitized publish, and duplicate-tap blocking
- flutter build web — production verification performed after evidence capture